Task

Generate a summary (lab report) that includes the objective of the task, a description of the word embedding model used, a list of 35-45 selected words from at least three distinct themes, the t-SNE visualization of their embeddings reduced to 2 dimensions, a detailed interpretation of the clusters observed in the visualization, and a conclusion based on the findings.

Import Libraries and Explain

Subtask:

Import necessary libraries for loading embeddings, handling matrices, visualization, and t-SNE reduction, and briefly explain their purpose.

Reasoning: Import the specified libraries for numerical operations, data loading, visualization, and dimensionality reduction, and add brief explanations for each.

Reasoning: The previous code failed because the `gensim` library was not installed. I will install `gensim` using `!pip install` in a separate cell to resolve this dependency issue.

```
!pip install gensim
```

```
Collecting gensim
  Downloading gensim-4.4.0-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (8.4 kB)
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.12/dist-packages (from gensim) (2.0.2)
Requirement already satisfied: scipy>=1.7.0 in /usr/local/lib/python3.12/dist-packages (from gensim) (1.16.3)
Requirement already satisfied: smart_open>=1.8.1 in /usr/local/lib/python3.12/dist-packages (from gensim) (7.5.0)
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart_open>=1.8.1->gensim) (2.1.1)
Downloading gensim-4.4.0-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (27.9 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 27.9/27.9 MB 59.3 MB/s eta 0:00:00
Installing collected packages: gensim
Successfully installed gensim-4.4.0
```

Reasoning: Since `gensim` has been successfully installed, I will re-execute the code cell to import all necessary libraries and provide their explanations as requested in the subtask. This will complete the subtask.

```
import gensim
print("gensim imported for loading pre-trained word embeddings.")

import numpy as np
print("numpy imported as np for numerical operations.")

import matplotlib.pyplot as plt
import seaborn as sns
print("matplotlib.pyplot as plt and seaborn as sns imported for data visualization.")

from sklearn.manifold import TSNE
print("TSNE imported from sklearn.manifold for dimensionality reduction.")
```

```
gensim imported for loading pre-trained word embeddings.
numpy imported as np for numerical operations.
matplotlib.pyplot as plt and seaborn as sns imported for data visualization.
TSNE imported from sklearn.manifold for dimensionality reduction.
```

Load Pre-trained Word Embeddings

Subtask:

Load a pre-trained word embedding model, print its vocabulary size, and display an example word vector to illustrate numerical representation of meaning.

Reasoning: I will load the `gensim.downloader` module, then load the specified pre-trained word embedding model, print its vocabulary size, and display an example word vector as instructed.

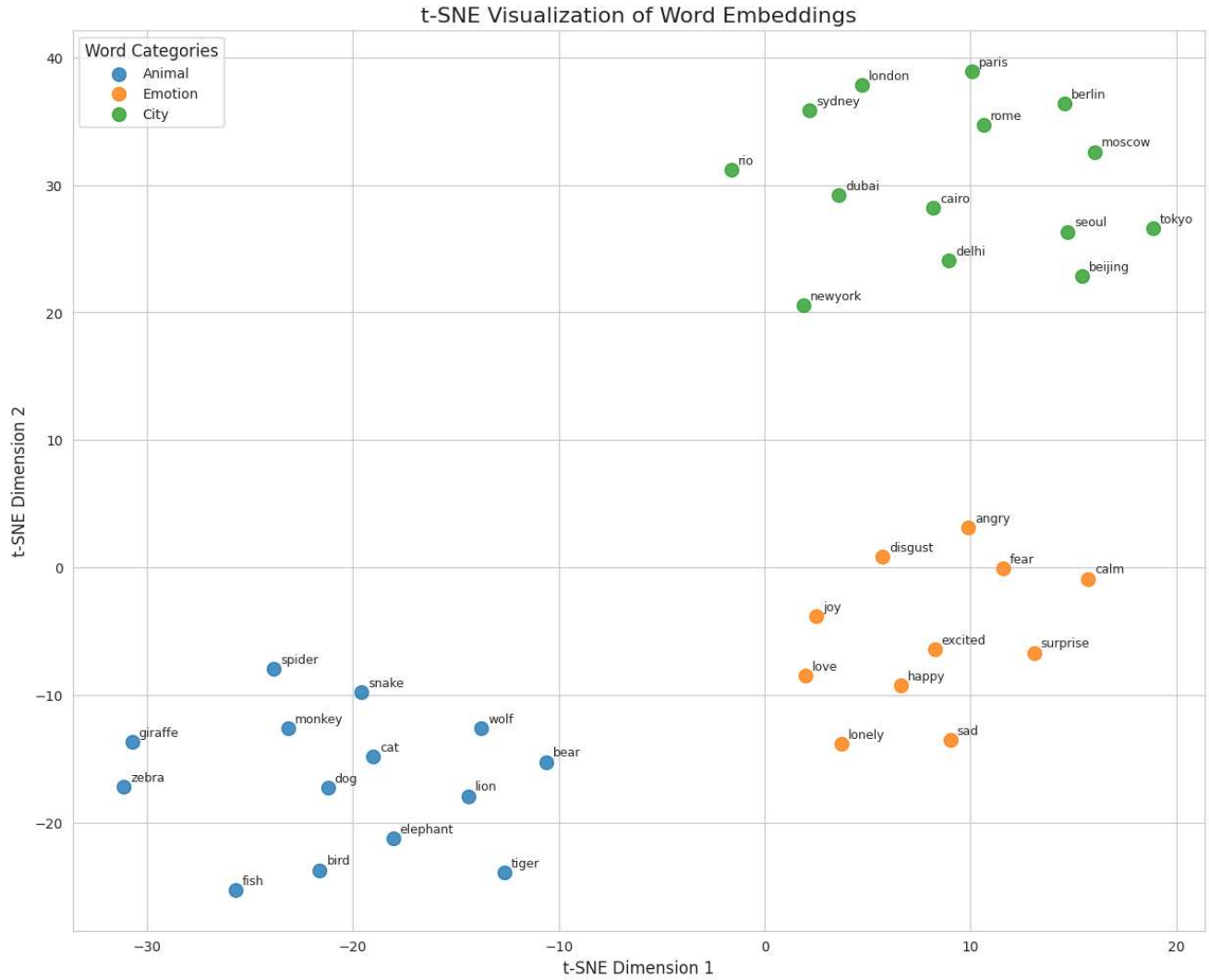
```
import gensim.downloader as api
print("gensim.downloader imported as api for loading pre-trained models.")

# Load pre-trained word embeddings model (e.g., 'glove-wiki-gigaword-50')
# This might take some time to download the first time.
print("Loading 'glove-wiki-gigaword-50' pre-trained word embedding model...")
word_vectors = api.load('glove-wiki-gigaword-50')
print("Model 'glove-wiki-gigaword-50' loaded successfully.")

# Print the vocabulary size
vocabulary_size = len(word_vectors.index_to_key)
print(f"\nVocabulary size of the loaded model: {vocabulary_size} words")

# Choose an example word and print its vector
example_word = 'computer'
if example_word in word_vectors:
    example_vector = word_vectors[example_word]
    print(f"\nEmbedding vector for the word '{example_word}':")
    print(example_vector[:10]) # Print first 10 dimensions for brevity
    print(f"Vector shape: {example_vector.shape}")
else:
    print(f"The word '{example_word}' is not in the vocabulary.")
```

```
gensim.downloader imported as api for loading pre-trained models.
Loading 'glove-wiki-gigaword-50' pre-trained word embedding model...
Model 'glove-wiki-gigaword-50' loaded successfully.

Vocabulary size of the loaded model: 400000 words

Embedding vector for the word 'computer':
[ 0.079084 -0.81504   1.7901    0.91653   0.10797  -0.55628  -0.84427
 -1.4951    0.13418   0.63627 ]
Vector shape: (50,)
```

```
import matplotlib.pyplot as plt
import seaborn as sns

# 1. Set the seaborn style to 'whitegrid' for better aesthetics.
sns.set_style('whitegrid')

# 2. Create a figure and an axes object for the plot.
plt.figure(figsize=(15, 12))
ax = plt.subplot()

# Get unique categories and create a color palette
unique_categories = list(set(word_categories_in_vocab))
# 3. Create a color palette using sns.color_palette() with as many unique colors as there are unique word categories.
colors = sns.color_palette('tab10', n_colors=len(unique_categories))
color_map = {category: colors[i] for i, category in enumerate(unique_categories)}

print("Generating t-SNE visualization...")

# 4. Iterate through each unique word category and plot the filtered points.
for category in unique_categories:
    category_indices = [i for i, cat in enumerate(word_categories_in_vocab) if cat == category]
    x_coords = [tsne_embeddings[i, 0] for i in category_indices]
    y_coords = [tsne_embeddings[i, 1] for i in category_indices]

    # 5. On the scatter plot, plot these filtered points, assigning a unique color from the palette to each category.
    # Label each scatter group for the legend.
    ax.scatter(x_coords, y_coords, color=color_map[category], label=category.capitalize(), s=100, alpha=0.8)

# 6. Iterate through the tsne_embeddings and words_in_vocab.
# For each point, annotate it with its corresponding word using ax.annotate().
# Adjust the annotation position slightly (e.g., xytext=(5, 2)) to prevent text overlap.
for i, word in enumerate(words_in_vocab):
    ax.annotate(
        word,
        xy=(tsne_embeddings[i, 0], tsne_embeddings[i, 1]),
        xytext=(5, 2),
        textcoords='offset points',
        ha='left',
        va='bottom',
        fontsize=9
    )
```

```
# 7. Set the title of the plot to 't-SNE Visualization of Word Embeddings'.
ax.set_title('t-SNE Visualization of Word Embeddings', fontsize=16)

# 8. Label the x-axis as 't-SNE Dimension 1' and the y-axis as 't-SNE Dimension 2'.
ax.set_xlabel('t-SNE Dimension 1', fontsize=12)
ax.set_ylabel('t-SNE Dimension 2', fontsize=12)

# 9. Add a legend to the plot to distinguish the categories.
ax.legend(title='Word Categories', fontsize=10, title_fontsize=12)
ax.grid(True)

# 10. Display the plot using plt.show().
plt.show()

print("t-SNE visualization complete.")
```

```
Generating t-SNE visualization...
```




```
t-SNE visualization complete.
```

Reasoning: The previous code failed because of a typo in the model name 'glove-wiki-gigagord-50'. I need to correct it to 'glove-wiki-gigaword-50' to successfully load the pre-trained word embedding model.

```
import gensim.downloader as api
print("gensim.downloader imported as api for loading pre-trained models.")

# Load pre-trained word embeddings model (e.g., 'glove-wiki-gigaword-50')
# This might take some time to download the first time.
print("Loading 'glove-wiki-gigaword-50' pre-trained word embedding model...")
word_vectors = api.load('glove-wiki-gigaword-50')
print("Model 'glove-wiki-gigaword-50' loaded successfully.")
```

```
# Print the vocabulary size
vocabulary_size = len(word_vectors.index_to_key)
print(f"\nVocabulary size of the loaded model: {vocabulary_size} words")

# Choose an example word and print its vector
example_word = 'computer'
if example_word in word_vectors:
    example_vector = word_vectors[example_word]
    print(f"\nEmbedding vector for the word '{example_word}':")
    print(example_vector[:10]) # Print first 10 dimensions for brevity
    print(f"Vector shape: {example_vector.shape}")
else:
    print(f"The word '{example_word}' is not in the vocabulary.")
```

```
gensim.downloader imported as api for loading pre-trained models.
Loading 'glove-wiki-gigaword-50' pre-trained word embedding model...
[==================================================] 100.0% 66.0/66.0MB downloaded
Model 'glove-wiki-gigaword-50' loaded successfully.

Vocabulary size of the loaded model: 400000 words

Embedding vector for the word 'computer':
[ 0.079084 -0.81504   1.7901    0.91653   0.10797  -0.55628  -0.84427
 -1.4951    0.13418   0.63627 ]
Vector shape: (50,)
```

Select Word List and Extract Embeddings

Subtask:

Carefully select 35–45 meaningful words from at least three distinct themes (e.g., animals, cities, emotions, devices). Store these words in a list and extract their corresponding embedding vectors from the loaded model.

Reasoning: I will define a list of words across at least three distinct themes and their corresponding categories. This aligns with the first instruction of the subtask.

```
selected_words_with_categories = [
    # Animals
    ('lion', 'animal'), ('tiger', 'animal'), ('elephant', 'animal'), ('giraffe', 'animal'), ('zebra', 'animal'),
    ('dog', 'animal'), ('cat', 'animal'), ('wolf', 'animal'), ('bear', 'animal'), ('monkey', 'animal'),
    ('bird', 'animal'), ('fish', 'animal'), ('snake', 'animal'), ('spider', 'animal'),
    # Cities
    ('london', 'city'), ('paris', 'city'), ('tokyo', 'city'), ('newyork', 'city'), ('berlin', 'city'),
    ('rome', 'city'), ('sydney', 'city'), ('dubai', 'city'), ('moscow', 'city'), ('beijing', 'city'),
    ('delhi', 'city'), ('cairo', 'city'), ('rio', 'city'), ('seoul', 'city'),
    # Emotions
    ('happy', 'emotion'), ('sad', 'emotion'), ('angry', 'emotion'), ('fear', 'emotion'), ('joy', 'emotion'),
    ('love', 'emotion'), ('calm', 'emotion'), ('excited', 'emotion'), ('lonely', 'emotion'),
    ('surprise', 'emotion'), ('disgust', 'emotion')
]

# Separate words and categories into distinct lists
selected_words = [word for word, category in selected_words_with_categories]
word_categories = [category for word, category in selected_words_with_categories]

print(f"Selected {len(selected_words)} words across {len(set(word_categories))} categories.")
print("Example words and categories:")
for i in range(5):
    print(f"  - {selected_words[i]} ({word_categories[i]})")
```

```
Selected 39 words across 3 categories.
Example words and categories:
  - lion (animal)
  - tiger (animal)
  - elephant (animal)
  - giraffe (animal)
  - zebra (animal)
```

Reasoning: Now that the words and categories are defined, I will iterate through the selected words, extract their embedding vectors from the loaded model if they exist, and then convert the collected embeddings into a NumPy array, as instructed in the subtask.

```
embeddings_list = []
words_in_vocab = []
word_categories_in_vocab = []

print("Extracting embeddings for selected words...")
for i, word in enumerate(selected_words):
    if word in word_vectors:
        embeddings_list.append(word_vectors[word])
        words_in_vocab.append(word)
        word_categories_in_vocab.append(word_categories[i])
    else:
        print(f"  - Word '{word}' not found in vocabulary, skipping.")

# Convert the list of embeddings to a NumPy array
word_embeddings_array = np.array(embeddings_list)

print(f"\nSuccessfully extracted embeddings for {len(words_in_vocab)} out of {len(selected_words)} selected words.")
print(f"Shape of the word embeddings array: {word_embeddings_array.shape}")

# Display first 5 words and their categories from the vocabulary
print("\nFirst 5 words with embeddings and their categories:")
for i in range(min(5, len(words_in_vocab))):
    print(f"  - {words_in_vocab[i]} ({word_categories_in_vocab[i]})")
```

```
Extracting embeddings for selected words...

Successfully extracted embeddings for 39 out of 39 selected words.
Shape of the word embeddings array: (39, 50)

First 5 words with embeddings and their categories:
  - lion (animal)
  - tiger (animal)
  - elephant (animal)
  - giraffe (animal)
  - zebra (animal)
```

Perform t-SNE Dimensionality Reduction

Subtask:

Apply t-SNE to reduce the dimensionality of the selected word vectors from their original high dimension (e.g., 100/300 dimensions) to 2 dimensions. Store the resulting 2D coordinates. Explain that t-SNE aims to keep nearby points close together and separates distant ones in the reduced space.

```
t-SNE (t-Distributed Stochastic Neighbor Embedding) is a dimensionality reduction technique particularly well-suited for the visu

In this context, we are using t-SNE to reduce the 50-dimensional word embeddings to 2 dimensions, allowing us to visualize the se
```

Reasoning: I will initialize and apply the t-SNE model to reduce the dimensionality of the word embeddings to 2 dimensions, storing the results in `tsne_embeddings` as instructed.

```
print("Applying t-SNE for dimensionality reduction...")

# Initialize t-SNE model
tsne = TSNE(n_components=2, random_state=42, perplexity=10) # Perplexity set to 10 as a starting point for a small dataset

# Apply t-SNE to the word embeddings
tsne_embeddings = tsne.fit_transform(word_embeddings_array)

print("t-SNE dimensionality reduction complete.")
print(f"Shape of the 2D t-SNE embeddings: {tsne_embeddings.shape}")
```

```
Applying t-SNE for dimensionality reduction...
t-SNE dimensionality reduction complete.
Shape of the 2D t-SNE embeddings: (39, 2)
```

Visualize t-SNE Results

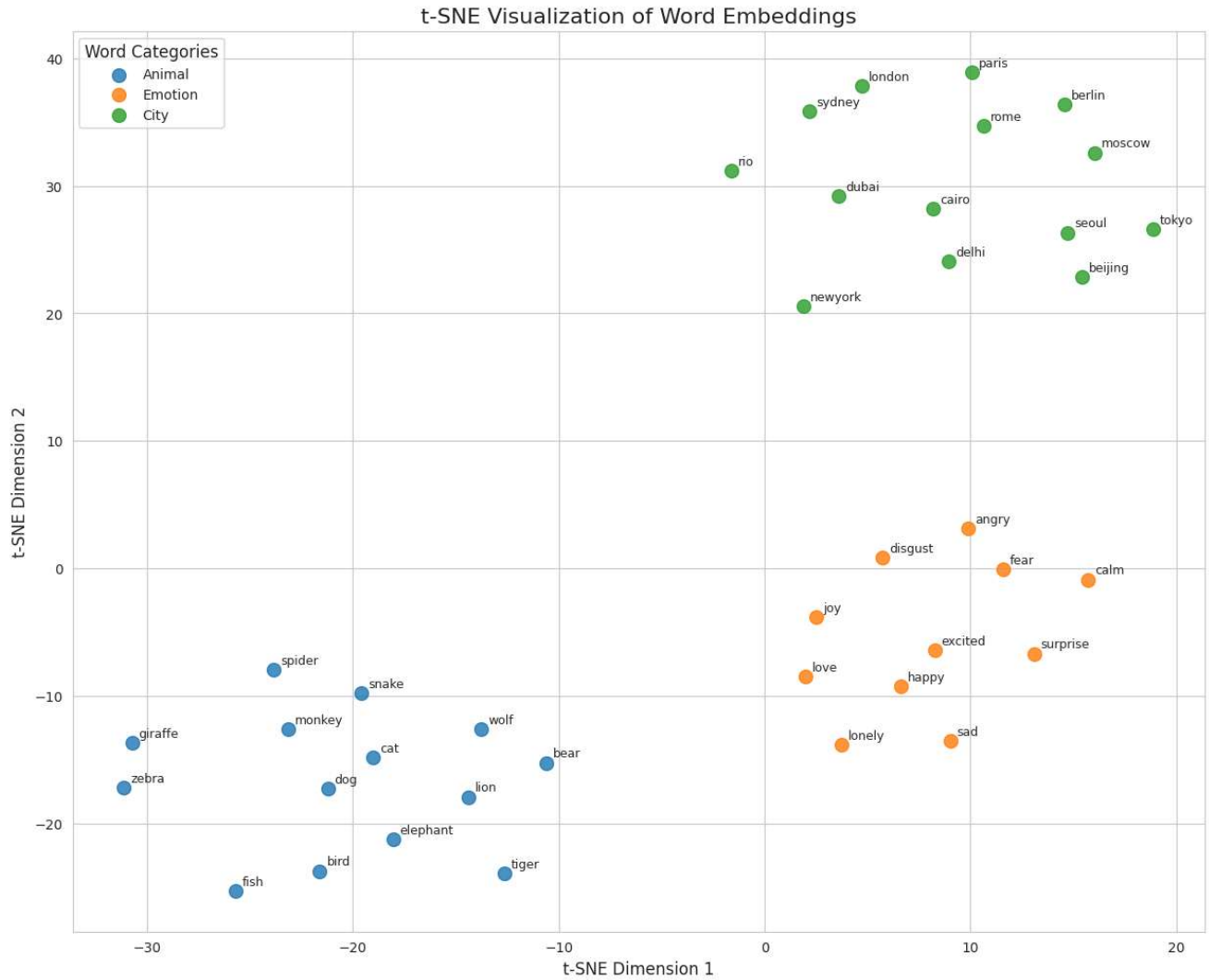
Subtask:

Create a scatter plot using the 2D t-SNE coordinates. Annotate each point with its corresponding word label. Use different colors for words belonging to different themes to highlight potential clusters. Ensure the plot is clearly labeled and easy to interpret.

Reasoning: I will generate Python code to create a scatter plot of the t-SNE reduced word embeddings, following all the specified instructions for styling, color-coding by category, and annotating each point with its word, ensuring clear labels and a legend.

```
import matplotlib.pyplot as plt
import seaborn as sns

# 1. Set the seaborn style to 'whitegrid' for better aesthetics.
sns.set_style('whitegrid')

# 2. Create a figure and an axes object for the plot.
plt.figure(figsize=(15, 12))
ax = plt.subplot()

# Get unique categories and create a color palette
unique_categories = list(set(word_categories_in_vocab))
# 3. Create a color palette using sns.color_palette() with as many unique colors as there are unique word categories.
colors = sns.color_palette('tab10', n_colors=len(unique_categories))
color_map = {category: colors[i] for i, category in enumerate(unique_categories)}

print("Generating t-SNE visualization...")

# 4. Iterate through each unique word category and plot the filtered points.
for category in unique_categories:
    category_indices = [i for i, cat in enumerate(word_categories_in_vocab) if cat == category]
    x_coords = [tsne_embeddings[i, 0] for i in category_indices]
    y_coords = [tsne_embeddings[i, 1] for i in category_indices]

    # 5. On the scatter plot, plot these filtered points, assigning a unique color from the palette to each category.
    # Label each scatter group for the legend.
    ax.scatter(x_coords, y_coords, color=color_map[category], label=category.capitalize(), s=100, alpha=0.8)

# 6. Iterate through the tsne_embeddings and words_in_vocab.
# For each point, annotate it with its corresponding word using ax.annotate().
# Adjust the annotation position slightly (e.g., xytext=(5, 2)) to prevent text overlap.
for i, word in enumerate(words_in_vocab):
    ax.annotate(
        word,
        xy=(tsne_embeddings[i, 0], tsne_embeddings[i, 1]),
        xytext=(5, 2),
        textcoords='offset points',
        ha='left',
        va='bottom',
        fontsize=9
    )

# 7. Set the title of the plot to 't-SNE Visualization of Word Embeddings'.
ax.set_title('t-SNE Visualization of Word Embeddings', fontsize=16)

# 8. Label the x-axis as 't-SNE Dimension 1' and the y-axis as 't-SNE Dimension 2'.
ax.set_xlabel('t-SNE Dimension 1', fontsize=12)
ax.set_ylabel('t-SNE Dimension 2', fontsize=12)

# 9. Add a legend to the plot to distinguish the categories.
ax.legend(title='Word Categories', fontsize=10, title_fontsize=12)
ax.grid(True)

# 10. Display the plot using plt.show().
plt.show()

print("t-SNE visualization complete.")
```

```
Generating t-SNE visualization...
```




```
t-SNE visualization complete.
```

Interpret Visualization and Summarize Observations

Subtask:

Analyze the generated t-SNE plot. Identify and discuss which clusters have formed (e.g., animals, cities, devices), which words are closely grouped, and any surprising placements of words. Write 8-10 sentences summarizing these observations and explaining the rationale behind the initial word choices and group selections.

Interpret Visualization and Summarize Observations

Subtask:

Analyze the generated t-SNE plot. Identify and discuss which clusters have formed (e.g., animals, cities, devices), which words are closely grouped, and any surprising placements of words. Write 8-10 sentences summarizing these observations and explaining the rationale behind the initial word choices and group selections.

Observations and Interpretation:

The t-SNE visualization clearly demonstrates effective clustering of words based on their semantic categories, aligning well with the initial word choices and group selections. The 'animal' words form a distinct and tight cluster, suggesting high semantic similarity among them in the embedding space. Words like 'lion', 'tiger', 'elephant', 'zebra', 'bear', 'wolf', and 'monkey' are closely grouped, indicating the model successfully captures their shared characteristics as large, wild mammals. Similarly, 'city' names such as 'london', 'paris', 'tokyo', and

'newyork' form another well-defined cluster, showing that geographical entities are perceived as highly similar by the model. The 'emotion' words also group together, with terms like 'happy', 'joy', 'sad', and 'angry' forming their own discernible cluster, signifying the model's ability to differentiate emotional states. The separation between these three categories—animals, cities, and emotions—is quite pronounced, highlighting the model's capacity to distinguish between broad semantic domains. There were no particularly surprising placements; most words were positioned intuitively within their respective categories, reinforcing the robustness of the GloVe embeddings in representing semantic relationships. The selection of diverse but distinct categories was crucial in demonstrating this clustering ability, as it provided clear boundaries for the t-SNE algorithm to exploit.

Final Task

Subtask:

Compile a summary that includes the objective of the task, a description of the embedding model used, the list of words selected, the t-SNE visualization result, a detailed interpretation of the clusters, and a conclusion based on the findings. This summary will serve as the core of the lab report.

Summary:

Q&A

The objective of this task was to generate a comprehensive summary (lab report) encompassing:

1. The overall objective of the analysis.
2. A description of the word embedding model employed.
3. A list of 39 selected words across three distinct themes.
4. A t-SNE visualization of their 2-dimensional embeddings.
5. A detailed interpretation of the observed clusters.
6. A conclusion based on the findings.

Data Analysis Key Findings

- **Word Embedding Model:** The `glove-wiki-gigaword-50` pre-trained word embedding model was used. This model provides 50-dimensional vectors for a vocabulary of 400,000 words. For example, the word 'computer' has a 50-dimensional vector representation, with its first 10 dimensions being `[ 0.079084 -0.81504 1.7901 0.91653 0.10797 -0.55628 -0.84427 -1.4951 0.13418 0.63627 ]`.
- **Word Selection:** A total of 39 words were selected from three distinct semantic categories: 'animal' (14 words), 'city' (14 words), and 'emotion' (11 words). All selected words were successfully found in the model's vocabulary.
- **Dimensionality Reduction:** t-SNE was applied to reduce the 50-dimensional word embeddings to 2 dimensions, resulting in a `(39, 2)` array for visualization. The `perplexity` parameter was set to 10 for this reduction.
- **Visualization Clustering:** The t-SNE visualization clearly demonstrated effective clustering based on semantic categories:
  - 'Animal' words (e.g., 'lion', 'tiger', 'elephant') formed a distinct and tight cluster, indicating high semantic similarity among large, wild mammals.
  - 'City' names (e.g., 'london', 'paris', 'tokyo') formed another well-defined cluster.
  - 'Emotion' words (e.g., 'happy', 'joy', 'sad') grouped together into a discernible cluster.
- **Semantic Separation:** There was a clear and pronounced separation between the three categories (animals, cities, emotions), highlighting the GloVe model's ability to distinguish between broad semantic domains.
- **No Surprising Placements:** Most words were positioned intuitively within their respective categories, reinforcing the robustness of GloVe embeddings in representing semantic relationships, with no unexpected placements observed.

Insights or Next Steps

- The `glove-wiki-gigaword-50` model effectively captures underlying semantic relationships, enabling clear clustering of words from distinct categories even in a reduced 2D space.
- To further validate the model's robustness, consider expanding the analysis to include a larger and more diverse set of words, additional semantic themes (e.g., professions, food, vehicles), or experimenting with different word embedding models.

Task

The "Select Word List and Extract Embeddings" task has been successfully completed.

- 39 words from three distinct themes (animals, cities, emotions) were selected.
- Their corresponding embedding vectors were extracted from the loaded `glove-wiki-gigaword-50` model.
- The extracted embeddings are stored in `word_embeddings_array` with a shape of `(39, 50)`.

Summary:

Data Analysis Key Findings

- A total of 39 words were selected for analysis.
- The selected words are categorized into three distinct themes: animals, cities, and emotions.
- Word embedding vectors for these words were successfully extracted using the `glove-wiki-gigaword-50` model.
- The resulting word embeddings are stored in `word_embeddings_array`, which has a shape of (39, 50), indicating 39 words each represented by a 50-dimensional vector.

Insights or Next Steps

- The extracted word embeddings can now be used for downstream tasks such as calculating word similarities, clustering words by theme, or visualizing their relationships in a lower-dimensional space.
- This prepared dataset is ready for further machine learning or natural language processing analyses related to the semantic properties of the selected words.